Sistemas de Tiempo Real — Práctica 3

Ejercicio 3: Banco de medición

Ingeniería en Computación

Entorno de medición

El circuito es el entorno común de la Parte II: un ATmega328P a 16 MHz con los pines PB0–PB3 llevados a los canales 0 a 3 del Logic Analyser, y la salida TXD del USART conectada a un Virtual Terminal a 9600 baudios. El firmware usa la biblioteca `Arduino_FreeRTOS` (feilipu) y se compiló con PlatformIO.

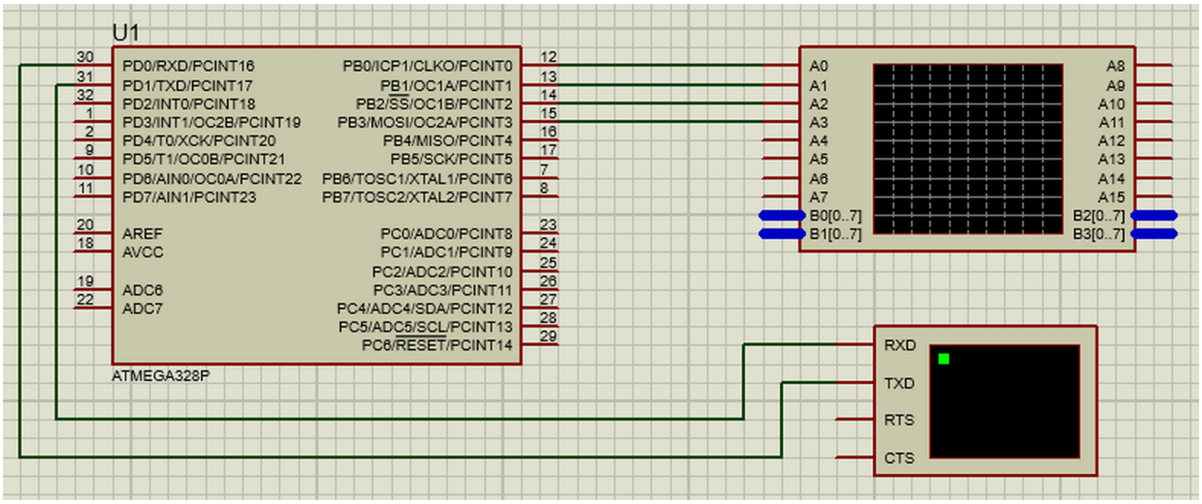


Figura 1: Banco de medición. PB0→A0, PB1→A1, PB2→A2, PB3→A3 y PD1/TXD al Virtual Terminal.

Escala de tiempos: desviación respecto del enunciado

El enunciado adopta 1 tick = 15 ms. El tick real de esta biblioteca lo genera el Watchdog Timer y vale `portTICK_PERIOD_MS` $= 2^4 = 16$ ms; no es configurable. Por eso se trabajó con 1 unidad = 4 ticks = 64 ms, de modo que los períodos y los tiempos de cómputo siguen siendo múltiplos exactos del tick, que es la condición que el enunciado busca garantizar. Para la tarea D ($C = 1$, $T = 3$) los valores teóricos son entonces $C = 64$ ms y $T = 192$ ms.

a) Por qué espera activa y no una llamada de bloqueo

Una llamada de bloqueo (`vTaskDelay`) saca la tarea de la lista de listos y cede el procesador: simula una tarea que espera una entrada/salida, un temporizador o un evento externo. No consume CPU, y por lo tanto no es lo que el parámetro C representa.

El lazo de espera activa mantiene la tarea en estado listo consumiendo ciclos de procesador. Eso es exactamente lo que significa C: demanda de CPU que el planificador tiene que servir. Solo así la tarea compite con las demás, es expropiada, y genera la interferencia que hace que el análisis de planificabilidad tenga sentido. Con una llamada de bloqueo el factor de utilización efectivo sería cero: nunca se perdería un plazo y la comparación entre Round Robin y FIFO de los Ejercicios 4 y 5 no mediría nada.

Hay una segunda condición, más fina. El lazo debe contar trabajo ejecutado, no tiempo de reloj. Se usó `_delay_loop_2`, que consume exactamente 4 ciclos por iteración: si la tarea es expropiada, el contador no avanza mientras no corre y la ráfaga se reanuda donde había quedado. Un lazo contra reloj de pared contaría como cómputo el tiempo que la tarea estuvo desalojada, y C se achicaría en cuanto hubiera competencia.

b) Medición del ancho de pulso y del período

Se compiló el firmware con una única tarea activa, la tarea D del Ejercicio 1 ($C = 1$, $T = 3$ unidades), que lleva PB3 a nivel alto mientras consume CPU y a nivel bajo al terminar la ráfaga. Una tarea de menor prioridad reporta por la terminal. La captura se tomó con *Capture Resolution* de 0,1 ms.

	Teórico	Medido	Error
Ancho de pulso ($C = 1$ unidad = 4 ticks)	64,00 ms	64,03 ms	0,05 %
Período ($T = 3$ unidades = 12 ticks)	192,00 ms	192,00 ms	0
Ciclo de trabajo	1/3	1/3	—
Plazos perdidos	0	0	—

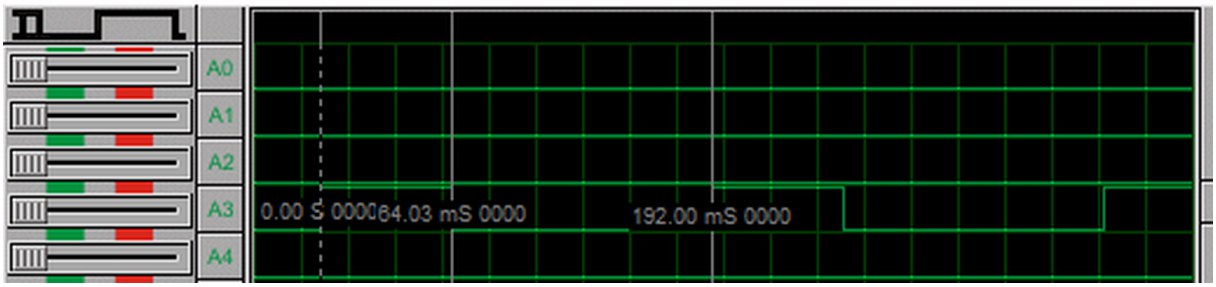


Figura 2: Canal A3 (PB3). Cursores sobre flanco de subida–bajada (64,03 ms) y subida–subida (192,00 ms). A0, A1 y A2 permanecen en bajo: hay una sola tarea activa.

La terminal confirma el mismo resultado por una vía independiente: cuenta ticks del propio programa, mientras el analizador mide tiempo real sobre el pin.

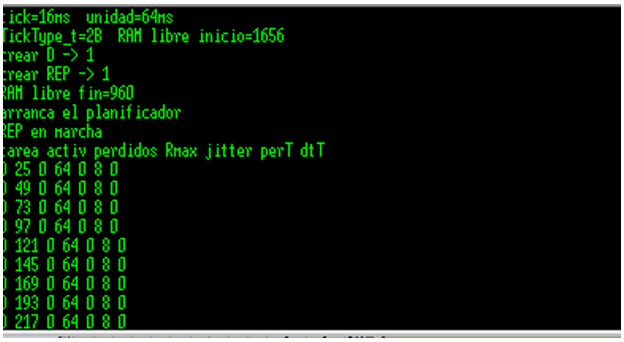


Figura 3: Reporte periódico. Columnas: tarea, activaciones, plazos perdidos, $R_{\text{máx}}$, jitter, ticks entre activaciones (`perT`) y ticks consumidos por la ráfaga (`dtT`).

Calibración: un defecto detectado y corregido. La primera versión del banco daba un período de 256 ms en lugar de 192. El reporte muestra por qué: la columna `dtT` vale 0, o sea que el contador de ticks no avanza durante la espera activa. En consecuencia `xTaskDelayUntil` no podía descontar el cómputo y agregaba el período completo después de la ráfaga, dejando un período efectivo de $C + T = 64 + 192 = 256$ ms, el valor observado.

La corrección consistió en descontar de la espera los ticks que la ráfaga debería haber consumido:

```
const TickType_t rafaga = t->C_unidades * TICKS_POR_UNIDAD;  /* 4 ticks  */
const TickType_t espera = periodo - rafaga;                  /* 12-4 = 8 */
...
xTaskDelayUntil(&prev, espera);
```

Las cuentas cierran en los dos sentidos: $64\,\text{ms} + 8 \times 16\,\text{ms} = 192\,\text{ms}$, y las activaciones entre reportes pasaron de 16 a 24, que es $192 \div 8$ con un reporte cada 192 ticks.

Un segundo defecto del banco. El bloque de reporte original superaba los 64 bytes del buffer de transmisión del USART. Al llenarse, `Serial.print` queda esperando dentro del controlador a que la interrupción lo vacíe, y el sistema se detenía por completo en el primer reporte. Se resolvió acortando cada línea a unos 15 caracteres. Es un caso típico de la materia: una tarea bloqueada dentro de un recurso compartido detiene al sistema entero.

Limitación del instrumento. Como `dtT` $= 0$, los tiempos que reporta la terminal tienen resolución de un tick y no observan lo que ocurre dentro de una ráfaga. La medición válida de C y del tiempo de respuesta es el ancho de pulso sobre el analizador lógico, que es el instrumento que el enunciado indica para la Parte II.

Alcance de este resultado. Conviene delimitarlo con cuidado, porque es la base del diseño de los Ejercicios 4 y 5. Lo medido aquí es que **durante una ráfaga de espera activa de 64 ms no se registra ningún tick**, cuando a 16 ms por tick deberían registrarse cuatro. En cambio el período completo da 192,00 ms exactos, lo que solo es posible si la espera de 8 ticks duró 128 ms, es decir si el tick vale 16 ms *en promedio* sobre el ciclo entero. Las dos cosas juntas significan que los ticks que no llegan durante el cómputo se recuperan durante el ocio.

Con una sola tarea y $U = 1/3$ sobra ocio y el balance cierra. El Ejercicio 4 carga las cuatro tareas, con $U = 0{,}9167$, y ahí el ocio deja de alcanzar: se mide allí que el período efectivo del tick pasa a 33–36 ms, más del doble del nominal. Esa es la razón por la que el banco de los Ejercicios 4 y 5 no puede apoyarse en el tick.

c) Expropiación a mitad de ráfaga

Sí: el tiempo total de CPU sigue siendo C. C es tiempo de procesador, no tiempo transcurrido. La expropiación no hace desaparecer trabajo: la tarea retoma donde quedó y todavía necesita los ciclos que le faltan. Esto vale

porque la ráfaga está implementada contando trabajo ejecutado, según lo discutido en el inciso a); con un lazo contra reloj de pared no se cumpliría.

Lo que sí crece es el tiempo de respuesta: $R = C + I$, donde I es la interferencia de las tareas que se intercalan. En este ejercicio ambos coinciden ($R_{\text{máx}} = C = 64\,\text{ms}$) porque no hay interferencia.

La verificación experimental está en el Ejercicio 4. Allí el pin de cada tarea lo escribe el propio núcleo en cada cambio de contexto, de modo que marca exactamente los tramos en que la tarea tuvo el procesador. Sumando los tramos de la primera ráfaga de cada tarea se obtiene:

Tarea	Tramos de ejecución (ms)	Suma
A	$14 + 2 + 16 + 16 + 16 + 16 + 11 + 5 + 16 + 16$	**128**
B	$16 + 13 + 3 + 16 + 16 + 16 + 16 + 11 + 5 + 16$	**128**
C	$16 + 16 + 13 + 3 + 16 + 16 + 16 + 16 + 10 + 6$	**128**
D	$16 + 16 + 16 + 12 + 4$	**64**

Las cuatro suman exactamente su tiempo de cómputo, repartido en hasta diez tramos. La expropiación cambia *cuándo* se ejecuta el trabajo, no cuánto hay.